PRD: Scalable Campus Resource Allocation System

1. Product Summary

This system manages hostel rooms and classrooms in a large university campus with frequent updates such as:

- room allocation
- classroom booking
- maintenance updates
- room swaps
- bulk reallocation
- availability queries

The backend is designed in **C++** and uses advanced tree-based data structures to answer queries fast and handle updates efficiently.

---

2. Goal

The system should support real-time campus operations for:

- **20,000+ students**
- **8 hostel blocks**
- **8,000+ rooms**
- **200+ classrooms**
- **daily updates and bulk changes**

The main goal is to make all common operations efficient, scalable, and easy to query.

---

3. Core Features

1. Add hostel rooms
2. Add classrooms
3. Query consecutive free rooms
4. Allocate rooms to students

5. Book classrooms for time slots
6. Query free classroom seats
7. Mark maintenance on floor/block/room range
8. Reallocate students when resources go offline
9. Swap room assignments
10. Release rooms or bookings
11. Query room/classroom status
12. Generate occupancy reports

---

4. Users

Primary user:

- **Admin**

Secondary user:

- **System operator / registrar backend**

This is not a student-facing product. The admin uses it to manage all resources.

---

5. Backend Technology Choice

Language

- **C++**

Why C++

- fast execution
- easy to implement segment trees and tree DP
- good memory control
- suitable for large-scale queries

---

6. System Data Model

6.1 Hostel Room

Each room stores:

- `roomId`
- `blockId`
- `floor`
- `capacity`
- `occupied`
- `status` (`FREE`, `ALLOCATED`, `MAINTENANCE`)
- `assignedStudents`

6.2 Classroom

Each classroom stores:

- `classId`
- `buildingId`
- `floor`
- `capacity`
- `timeSchedule`
- `status`

6.3 Student

Each student stores:

- `studentId`
- `roomId`
- `blockId`
- `floor`
- `status`

---

7. Input/Output Format

The backend will work as a **single command processor** using standard input and standard output.

7.1 General Format

Input

Q
COMMAND_1
COMMAND_2
...
COMMAND_Q

Output

Each command produces one response line, or multiple response lines if needed.

---

8. Command Specification

8.1 Add a hostel room

Command

ADD_ROOM <roomId> <blockId> <floor> <capacity>

Example

ADD_ROOM R101 C 3 4

Output

OK RoomAdded

Backend logic

- validate roomId is unique
- create room record
- insert it into:
    - room map

    - block-wise room list
    - floor-wise segment tree base array
- initialize room as free

8.2 Add a classroom

Command

ADD_CLASSROOM <classId> <buildingId> <floor> <capacity>

Example

ADD_CLASSROOM CL12 B1 2 60

Output

OK ClassroomAdded

Backend logic

- validate classId uniqueness
- insert classroom into classroom map
- initialize time-slot structure for booking
- mark all seats free for all time slots

8.3 Query consecutive free rooms in a block

Command

QUERY_FREE_ROOMS <blockId> <count>

Example

QUERY_FREE_ROOMS C 50

Output

FOUND R301 R350

or

NOT_FOUND

Backend logic

- use segment tree on the block's room array
- each node stores:
  - total free count
  - prefix free length
  - suffix free length
  - best consecutive free length
- search for a segment of size `count`
- return first valid consecutive range

Complexity

- Query: `O(log n)`

8.4 Allocate rooms to students

Command

ALLOCATE_ROOMS <studentCount> <preferredBlock> <minCapacity>

Example

ALLOCATE_ROOMS 300 C 3

Output

ALLOCATED 300
ROOMS R301 R302 R303 ...

or

PARTIAL 240 60

Backend logic

- search preferred block first
- use consecutive-free search and room capacity check
- allocate rooms in greedy order
- update segment tree values
- mark rooms occupied
- if not enough rooms, return partial allocation

Allocation rules

- prefer same block
- prefer lower floors first
- prefer closer room ranges
- avoid maintenance rooms

Complexity

- `O(k log n)` where `k` is number of rooms assigned

8.5 Mark room / floor / block maintenance

Command

MAINTENANCE <scopeType> <scopeId> <status>

Allowed values

- `scopeType = ROOM`
- `scopeType = FLOOR`
- `scopeType = BLOCK`
- `status = ON | OFF`

Examples

MAINTENANCE ROOM R101 ON
MAINTENANCE FLOOR C 3 ON
MAINTENANCE BLOCK D ON

Output

OK MAINTENANCE_UPDATED 42

Backend logic

- if scope is room, update one room
- if floor/block, apply range update to all rooms in that scope
- use lazy propagation in segment tree to mark many rooms at once
- if `ON`, mark unavailable
- if `OFF`, restore based on prior occupancy state rules

Important rule

If a room is already occupied, turning maintenance ON does not delete the student automatically. It marks the room as blocked for future allocation and triggers reallocation logic only if requested.

Complexity

- `O(log n)` for range update

8.6 Release room

Command

RELEASE_ROOM <roomId>

Example

RELEASE_ROOM R101

Output

OK RELEASED

Backend logic

- verify room exists
- verify room is currently allocated

- clear student assignment
- update room status to free unless under maintenance
- update segment tree node values

8.7 Swap two room assignments

Command

SWAP_ROOMS <studentId1> <studentId2>

Example

SWAP_ROOMS S12 S34

Output

OK SWAPPED

Backend logic

- find room assignments of both students
- verify both rooms are valid and not under maintenance
- swap the student lists
- update internal student-to-room mapping
- update room metadata

Complexity

- `O(log n)` or `O(1)` depending on storage structure

8.8 Book classroom seats for a time interval

Command

BOOK_CLASSROOM <classId> <startTime> <endTime> <seats>

Time format

- `HH:MM`
- example: `14:00`, `17:30`

Example

BOOK_CLASSROOM CL12 14:00 17:00 30

Output

BOOKED

or

NOT_ENOUGH_SEATS

Backend logic

- classroom has a time-based availability structure
- use segment tree over time slots or discrete time buckets
- check if requested seats are available for the full interval
- if yes, reserve seats for that time range
- update schedule

Complexity

- `O(log T)` where `T` is number of time slots

8.9 Query free seats in classrooms

Command

QUERY_FREE_SEATS <startTime> <endTime> <buildingId>

Example

QUERY_FREE_SEATS 14:00 17:00 B1

Output

FREE_SEATS 240

Backend logic

- iterate classroom tree or classroom index for the requested building
- query each classroom's availability for the given interval
- sum the free seats
- return total

Optimization

- maintain building-wise aggregated segment tree
- return in `O(log n)` or `O(log n + k)` depending on design

8.10 Reallocate students from a floor or block

Command

REALLOCATE <scopeType> <scopeId>

Example

REALLOCATE FLOOR C_3
REALLOCATE BLOCK D

Output

REALLOCATED 80 FAILED 10

Backend logic

- identify all students affected by the scope
- collect available rooms in other blocks/floors
- sort target rooms by:
    - same block first
    - nearest floor next
    - highest availability next
- assign students using greedy + tree query
- if not enough room, leave some failed

Data structure use

- tree DP can help choose best target block/floor
- segment tree helps find free ranges fast

Complexity

- depends on count of students affected
- typically `O(k log n)`

8.11 Query room status

Command

ROOM_STATUS <roomId>

Example

ROOM_STATUS R101

Output

ROOM R101 BLOCK C FLOOR 3 CAPACITY 4 STATUS FREE OCCUPIED 0

Backend logic

- read room record from hash map
- return current metadata

8.12 Query classroom status

Command

CLASSROOM_STATUS <classId>

Example

CLASSROOM_STATUS CL12

Output

CLASSROOM CL12 BUILDING B1 FLOOR 2 CAPACITY 60 STATUS ACTIVE

Backend logic

- fetch from classroom map
- print current usage and availability state

8.13 Generate occupancy report

Command

REPORT <scopeType> <scopeId>

Example

REPORT BLOCK C
REPORT FLOOR C_3
REPORT CAMPUS ALL

Output

TOTAL_ROOMS 120
FREE 40
ALLOCATED 70
MAINTENANCE 10

Backend logic

- aggregate counts from segment tree nodes and maps
- produce summary report for admin

---

9. Request Processing Flow in C++

Every request follows this backend pipeline:

1. Read command
2. Parse command name and arguments
3. Validate input
4. Locate target object
5. Run query/update on tree structure
6. Update maps and metadata
7. Return response string

---

10. Recommended Backend Modules in C++

10.1 `RoomManager`

Handles:

- add room
- allocate room
- release room
- maintenance update
- room status

10.2 `ClassroomManager`

Handles:

- add classroom
- booking
- seat availability
- classroom status

10.3 `BlockTree`

Handles:

- block-wise segment tree
- consecutive free room queries

- bulk maintenance update

10.4 `TimeTree`

Handles:

- classroom schedule by time slot
- time interval booking
- free-seat queries

10.5 `Allocator`

Handles:

- room selection
- student assignment
- reallocation
- swap logic

10.6 `Reporter`

Handles:

- occupancy summaries
- admin dashboards
- block/floor statistics

---

11. Data Structures to Use

11.1 Segment Tree

Used for:

- consecutive free rooms
- range availability queries
- fast updates

11.2 Lazy Propagation

Used for:

- floor maintenance
- block maintenance
- bulk state changes

11.3 Tree DP

Used for:

- choosing best allocation target
- minimizing movement during reallocation
- picking nearest available zone

11.4 Hash Maps

Used for:

- roomId → room object
- classId → classroom object
- studentId → assigned room

11.5 Ordered Sets / Vectors

Used for:

- maintaining room order
- fast nearest-room search
- floor-wise storage

---

12. Backend Logic Rules

Room allocation priority

1. same block
2. same floor
3. nearest floor

4. first available consecutive range

Classroom booking rule

- booking succeeds only if full time interval has enough seats

Maintenance rule

- maintenance ON makes rooms unavailable for new allocation
- maintenance OFF restores availability if the room is not occupied

Reallocation rule

- affected students should be moved to nearest valid free rooms
- minimize total movement across campus

---

13. Edge Cases

The system must handle:

- invalid room/classroom IDs
- duplicate IDs
- zero or negative capacity
- booking outside valid time
- maintenance on an already maintained room
- allocation request larger than available capacity
- overlapping bookings
- releasing an already free room
- querying an empty block

---

14. Error Responses

Use consistent errors:

ERR_INVALID_ID
ERR_ALREADY_EXISTS

ERR_NOT_FOUND
ERR_NOT_AVAILABLE
ERR_NOT_ENOUGH_CAPACITY
ERR_INVALID_TIME
ERR_INVALID_REQUEST

---

15. Sample Input and Output

Input

8
ADD_ROOM R101 C 3 4
ADD_ROOM R102 C 3 4
ADD_ROOM R103 C 3 4
QUERY_FREE_ROOMS C 2
ALLOCATE_ROOMS 2 C 4
ROOM_STATUS R101
MAINTENANCE ROOM R102 ON
REPORT BLOCK C

Output

OK RoomAdded
OK RoomAdded
OK RoomAdded
FOUND R101 R102
ALLOCATED 2
ROOM R101 BLOCK C FLOOR 3 CAPACITY 4 STATUS ALLOCATED OCCUPIED 1
OK MAINTENANCE_UPDATED 1
TOTAL_ROOMS 3 FREE 1 ALLOCATED 1 MAINTENANCE 1

---

16. Complexity Targets

Operation	Target Complexity

Add room/classroom	`O(1)` to `O(log n)`
Query free rooms	`O(log n)`
Allocate rooms	`O(k log n)`
Maintenance update	`O(log n)`
Classroom booking	`O(log T)`
Query seats	`O(log n)`
Reallocation	`O(k log n)`

---

17. Implementation Order in C++

Best implementation order:

1. room and classroom classes
2. maps for lookup
3. segment tree for room availability
4. lazy propagation for maintenance
5. classroom time booking tree
6. allocation logic
7. swap and reallocation
8. reports and edge handling